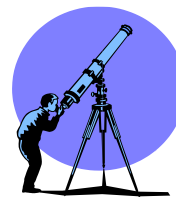


# Automata, Languages and Computing

## Teoria della complessità La risorsa SPAZIO



1

1

## Teoria della complessità

- obiettivo
  - classificazione dei problemi dal punto di vista delle risorse di calcolo che richiedono
  - identificazione di classi di complessità
- risorsa
  - spazio (memoria)
- modello di calcolo:
  - macchina di Turing
    - ipotesi di notevole generalità

2

2

## La risorsa spazio – determinismo

- sia  $M$  una *MT deterministica* che decide un linguaggio
  - cioè si ferma su ogni possibile input
- la *complessità spaziale* di  $M$  è la funzione  $s: \mathbb{N} \rightarrow \mathbb{N}$ , dove  $s(n)$  è il massimo numero di celle di nastro a cui  $M$  accede per ogni input di lunghezza  $n$

3

3

## La risorsa spazio – non determinismo

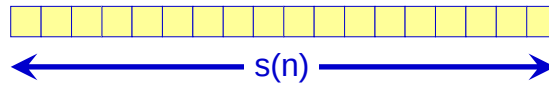
- sia  $M$  una *MT non deterministica* che decide un linguaggio
  - cioè si ferma su ogni possibile input
- la *complessità spaziale* di  $M$  è la funzione  $s: \mathbb{N} \rightarrow \mathbb{N}$ , dove  $s(n)$  è il massimo numero di celle di nastro a cui  $M$  accede in ogni sua computazione per ogni input di lunghezza  $n$

4

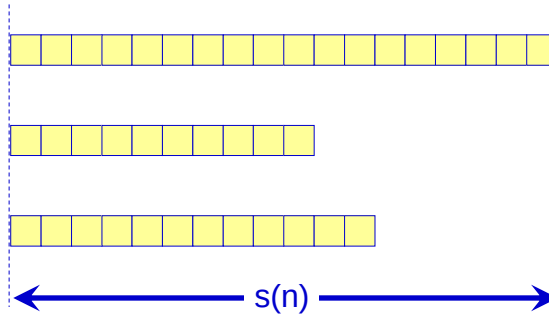
4

## Spazio deterministico e non deterministico

spazio  
deterministico



spazio non  
deterministico



5

5

## Complessità spaziale

Primi risultati

6

6

## Complessità e problemi di decisione su linguaggi

- data una funzione  $s: \mathbb{N} \rightarrow \mathbb{N}$  possiamo definire le seguenti classi di linguaggi, con riferimento a MT ad un nastro
- $DSPACE(s(n))$ 
  - insieme dei linguaggi decisi da una MT *deterministica* utilizzando spazio  $O(s(n))$
- $NSPACE(s(n))$ 
  - insieme dei linguaggi decisi da una MT *non deterministica* utilizzando spazio  $O(s(n))$

7

7

## SAT è decidibile in spazio lineare deterministico

**teorema:**  $SAT \in DSPACE(n)$

**dimostrazione:**

- sia  $n$  la lunghezza dell'input
  - le variabili sono quindi al più  $n$
- generiamo sul nastro, in iterazioni successive, tutte le possibili assegnazioni vero/falso delle variabili
- in ogni iterazione valutiamo se la formula è vera e cancelliamo il nastro, per riusarlo nell'iterazione successiva
- l'unica informazione da mantenere sul nastro è il valore corrente vero/falso delle variabili

8

8

## Relazioni elementari tra classi di complessità

**teorema:** per ogni  $f: \mathbb{N} \rightarrow \mathbb{N}$  con  $f(n) \geq n$

$\text{DTIME}(f(n)) \subseteq \text{NTIME}(f(n))$  e

$\text{DSpace}(f(n)) \subseteq \text{NSpace}(f(n))$

**dimostrazione:** una MT è una MTND

**teorema:** per ogni  $f: \mathbb{N} \rightarrow \mathbb{N}$  con  $f(n) \geq n$

$\text{DTIME}(f(n)) \subseteq \text{DSpace}(f(n))$  e

$\text{NTIME}(f(n)) \subseteq \text{NSpace}(f(n))$

**dimostrazione:** in  $t$  passi di computazione una MT può usare al più  $t$  celle di nastro

9

9

## Tempo e spazio deterministici

**lemma:** se una MT decide un linguaggio usando spazio  $s(n)$ , con  $s(n) \geq n$ , allora le sue computazioni terminano in tempo  $2^{O(s(n))}$

**dimostrazione:**

- consideriamo il comportamento della MT  $M$  sulla stringa  $x$  con  $|x|=n$
- siano  $c_1, c_2, \dots, c_t$  le configurazioni attraversate da  $M$ , con  $c_i$  configurazione al tempo  $i$  e  $t$  tempo di calcolo di  $M$  su  $x$
- nella sequenza  $c_1, c_2, \dots, c_t$  non possono esserci due  $c_i$  uguali, altrimenti  $M$  ciclerebbe

10

10

## Tempo e spazio deterministici

### dimostrazione (continua):

- il numero di possibili configurazioni attraversate è quindi al più il prodotto tra: il numero di stati, il numero di possibili posizioni della testina sul nastro, il numero di possibili contenuti delle celle

– tale numero è

$$O(1) \cdot s(n) \cdot |\Sigma|^{s(n)} = 2^{O(s(n)) + \log s(n)} = 2^{O(s(n))}$$

– infatti

$$O(1) \cdot s(n) = O(s(n)) = 2^{\log O(s(n))} = 2^{O(\log s(n))}$$

– inoltre, per una qualche costante  $p$ ,  $|\Sigma|=2^p$ , quindi

$$|\Sigma|^{s(n)} = 2^{p \cdot s(n)} = 2^{O(s(n))}$$

11

11

## Tempo e spazio deterministici

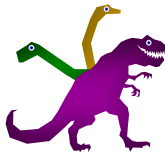
### dimostrazione (continua):

- dato che  $M$  non può passare per una configurazione due volte durante la computazione, la computazione deve finire in  $2^{O(s(n))}$  passi

12

12

## Teorema di Savitch



13

13

## Teorema di Savitch

**teorema:** per ogni  $s: \mathbb{N} \rightarrow \mathbb{N}$  con  $s(n) \geq n$ ,  
 $\text{NSPACE}(s(n)) \subseteq \text{DSPACE}(s^2(n))$

**dimostrazione:**

- simuliamo deterministicamente la MTND che usa spazio  $s(n)$
- una strategia troppo semplice non funziona
  - infatti potremmo provare una ad una tutte le computazioni non deterministiche
  - però in questo caso in ogni computazione dobbiamo ricordare quale essa sia, per poter passare alla successiva

14

14

## Teorema di Savitch

- una strategia troppo semplice non funziona (continua):
  - ma una computazione che usa spazio  $O(s(n))$  può attraversare  $2^{O(s(n))}$  diverse configurazioni, facendo una scelta non deterministica per ciascuna di esse
  - questo potrebbe portarci a dover usare spazio  $2^{O(s(n))}$ 
    - molto di più di quanto dica il teorema
- quindi cambiamo strategia, studiando un problema correlato, lo *yieldability problem*

15

15

## Lo yieldability problem

- sono date due configurazioni  $c_1$  e  $c_2$  di una MTND  $N$  ed un numero  $T$
- vogliamo verificare se  $N$  può raggiungere  $c_2$  da  $c_1$  in al più  $T$  passi
- lo yieldability problem può essere risolto da una MT deterministica
  - si prova con ogni configurazione intermedia  $c_m$  e si verifica ricorsivamente se da  $c_1$  si può raggiungere  $c_m$  e da  $c_m$  si può raggiungere  $c_2$ , in entrambi i casi con al più  $T/2$  passi
  - il secondo test può riusare lo spazio del primo

16

16



## Lo yieldability problem

- attenzione: c'è bisogno di spazio ulteriore per memorizzare lo stack della ricorsione
- ogni livello della ricorsione richiede spazio  $O(s(n))$  per memorizzare la configurazione
- la profondità della ricorsione è  $\log(T)$ , dove  $T$  è il tempo impiegato dalla computazione più profonda della MTND
- abbiamo che  $T=2^{O(s(n))}$ , quindi  $\log(T)=O(s(n))$
- quindi la computazione deterministica usa spazio  $O(s^2(n))$

17

17

## Teorema di Savitch

**dimostrazione (reloaded):**

- sia  $N$  una MTND che decide il linguaggio  $A$  con spazio  $O(s(n))$
- costruiamo una MT  $M$  che decide  $A$
- $M$  usa una MT *canyield* che verifica se da una configurazione se ne può raggiungere un'altra in un numero di passi prefissato

18

18

## Teorema di Savitch

### dimostrazione (continua):

- data la stringa  $w$  di input, con  $|w|=n$ , le configurazioni  $c_1$  e  $c_2$  di  $N$  su  $w$  e  $T$ , la MT  $\text{canyield}(c_1, c_2, T)$  accetta se, quando  $N$  parte dalla configurazione  $c_1$ , la stessa  $N$  ha una qualche computazione che la porta in  $c_2$  in al più  $T$  passi; altrimenti rifiuta

19

19

## Teorema di Savitch

### dimostrazione:

- $\text{canyield}(c_1, c_2, T)$ 
  - se  $T=1$  verifica se  $c_1=c_2$  o se  $c_2$  è raggiungibile in un passo; se si accetta, altrimenti rifiuta
  - se  $T>1$ , per ogni configurazione  $c_m$  di  $N$  su  $w$ , che usa spazio  $s(n)$ 
    - esegui  $\text{canyield}(c_1, c_m, T/2)$ , esegui  $\text{canyield}(c_m, c_2, T/2)$
    - se entrambe accettano allora accetta, altrimenti rifiuta

20

20

## Teorema di Savitch

### dimostrazione:

- costruiamo  $M$  come segue
- modifichiamo  $N$  in modo che abbia una sola configurazione di accettazione  $c_F$
- sia  $c_0$  la configurazione iniziale di  $N$  su  $w$
- scegliamo una costante  $d$  tale che  $N$  non abbia più di  $2^{ds(n)}$  configurazioni usando nastro  $s(n)$
- osserva:  $2^{ds(n)}$  limita superiormente il tempo impiegato da ogni *branch* di  $N$

21

21

## Teorema di Savitch

### dimostrazione:

- $M$  restituisce il risultato di  $\text{canyield}(c_0, c_F, 2^{ds(n)})$
- la simulazione è, per quanto detto in precedenza, corretta
- ogni volta che  $\text{canyield}$  viene invocata memorizza  $c_1$ ,  $c_2$  e  $T$  sul nastro in uno stack
- ogni livello di ricorsione usa spazio addizionale  $O(s(n))$

22

22

## Teorema di Savitch

### dimostrazione:

- inizialmente  $T=2^{ds(n)}$
- ogni livello di ricorsione divide  $T$  per 2
- la profondità della ricorsione è  

$$O(\log(2^{ds(n)}))=O(s(n))$$
- lo spazio usato è effettivamente  $s^2(n)$

23

23

## Teorema di Savitch

### dimostrazione:

- una questione sottile
  - quando  $M$  esegue canyieid deve conoscere  $s(n)$
  - per risolvere il problema possiamo modificare  $M$  in modo che tenti i valori di  $s(n)$  ad uno ad uno:  
 $1,2,3,4,5,6,7,8,\dots$
  - la macchina smette di tentare quando non raggiunge nessuna configurazione di una certa dimensione
  - certo come tempo di calcolo non è il massimo ma ....

24

24

## PSPACE

25

25

### La classe PSPACE

$$\text{PSPACE} = \bigcup_{k=0}^{\infty} \text{DSPACE}(n^k)$$

$$\text{NPSPACE} = \bigcup_{k=0}^{\infty} \text{NSPACE}(n^k)$$

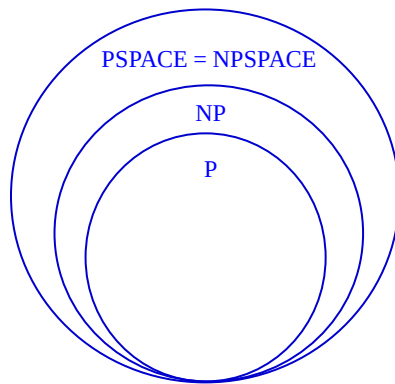
il teorema di Savitch implica che:

$$\text{PSPACE} = \text{NPSPACE}$$

26

26

## Relazioni tra classi di complessità



problemi ancora aperti

$P = NP ?$

$PSPACE = NP ?$

ci si convince sempre di più  
che non sia così, ma nessuno  
lo ha mai dimostrato

27

27

## Un problema in PSPACE: TQBF

- QBF (Fully Quantified Boolean Formula)  
=  $\{\Phi \mid \Phi \text{ è una formula booleana pienamente quantificata vera}\}$
- *definizione: formula booleana pienamente quantificata*
  - formula booleana
    - le variabili possono essere vere o false
  - pienamente quantificata
    - con quantificatori (esistenziali o universali) per ogni variabile, tutti all'inizio della formula
  - può contenere costanti (true o false)

28

28

## Un problema in PSPACE: TQBF

problema di decisione TQBF (True Fully Quantified Boolean Formula)

- istanza
  - una formula booleana pienamente quantificata  $\Phi$
- predicato
  - $\Phi$  è vera

29

29

## Un problema in PSPACE: TQBF

- es:  $\Phi = \forall x \exists y [(x \vee y) \wedge (\neg x \vee \neg y)]$ 
  - in questo caso  $\Phi$  è vera, per stabilirlo ragioniamo come segue
  - il primo  $\forall$  richiede che  $\Phi$  sia vera per ogni scelta di  $x$  e quindi per  $x=\text{true}$  e per  $x=\text{false}$ 
    - caso  $x=\text{true}$ , è vero che  $\exists y[(\text{true} \vee y) \wedge (\text{false} \vee \neg y)]$ ? sì, basta scegliere  $y=\text{false}$
    - caso  $x=\text{false}$ , è vero che  $\exists y[(\text{false} \vee y) \wedge (\text{true} \vee \neg y)]$ ? sì, basta scegliere  $y=\text{true}$

30

30

## Un problema in PSPACE: TQBF

- es:  $\Phi' = \exists x \forall y [(x \vee y) \wedge (\neg x \vee \neg y)]$ 
  - $\Phi'$ , ottenuta scambiando i quantificatori nell'esempio precedente, è falsa
  - per stabilirlo ragioniamo come segue
  - il primo  $\exists$  richiede che il resto della formula sia vero per almeno uno tra  $x=\text{true}$  e  $x=\text{false}$ 
    - caso  $x=\text{true}$ , è vero che  $\forall y [(true \vee y) \wedge (false \vee \neg y)]$ ? no, non è vero per  $y=\text{true}$
    - caso  $x=\text{false}$ , è vero che  $\forall y [(false \vee y) \wedge (true \vee \neg y)]$ ? no, non è vero per  $y=\text{false}$
    - quindi il resto della formula non è vero per nessuna delle due possibili scelte di  $x$

31

31

## Spazio polinomiale e TQBF

**teorema:**  $TQBF \in PSPACE$

**dimostrazione:** algoritmo Test

- se  $\Phi$  contiene solo costanti è facile valutarla e restituire accetta o rifiuta
- se  $\Phi$  è nella forma  $\exists x \Psi$  chiama ricorsivamente Test su  $\Psi$ , prima con  $x=\text{true}$  e poi con  $x=\text{false}$ , se uno dei risultati è accetta allora accetta, altrimenti rifiuta
- se  $\Phi$  è nella forma  $\forall x \Psi$  chiama ricorsivamente Test su  $\Psi$ , prima con  $x=\text{true}$  e poi con  $x=\text{false}$ , se entrambi i risultati sono accetta allora accetta, altrimenti rifiuta

32

32



## Spazio polinomiale e TQBF

dimostrazione:

- per ciò che riguarda lo spazio richiesto:
  - la profondità della ricorsione è al più pari al numero di variabili
  - ad ogni livello dobbiamo memorizzare il valore di una variabile
  - quindi lo spazio usato è  $O(n)$ , con  $n$  numero di variabili

33

33

## Perché è difficile costruire un verificatore polinomiale per TQBF?

- se ci riuscissimo vorrebbe dire che  $TQBF \in NP$
- ma come dovrebbe essere fatto un certificato verificabile in tempo polinomiale deterministico?
- intuitivamente qui il problema è quello di dover verificare ciò che accade per tutte le combinazioni di scelte possibili per i valori delle variabili
- per SAT era più facile, bastava mostrare un valore per ciascuna variabile

34

34

## PSPACE-completezza

- anche per la classe PSPACE è possibile definire una completezza
  - obiettivo: identificare i problemi più difficili della classe
- come per NP, anche in questo caso le riduzioni usate sono riduzioni che impiegano tempo polinomiale
- perché non usare riduzioni che usano spazio polinomiale?

35

35

## PSPACE-completezza

- regola generale per definire la completezza rispetto ad una classe:
  - quando si definiscono i problemi completi rispetto ad una classe il modello di calcolo usato per la riduzione deve essere meno potente di quello usato per definire la stessa classe

36

36

## PSPACE-completezza

teorema: TQBF è PSPACE completo

37

37

## PSPACE e le strategie per giocare

38

38

## Il gioco della geografia

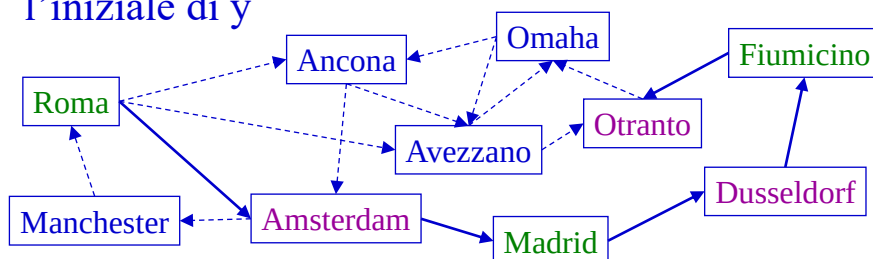
- due giocatori **A** e **B** a turno scrivono nomi di città
  - il nome di una città deve iniziare con l'ultima lettera della città precedente
  - una città non può essere ripetuta
  - perde il giocatore che non trova il nome di una città da scrivere (e vince l'altro giocatore)
- es. **A**-Roma, **B**-Amsterdam, **A**-Madrid, **B**-Dusseldorf, **A**-Fiumicino, **B**-Otranto, .... finché uno dei due non riesce più a trovare una città da scrivere e quindi perde

39

39

## Il gioco della geografia

- possiamo descrivere il gioco della geografia come un grafo; i vertici sono le città; c'è un arco tra la città  $x$  e la città  $y$  se l'ultima lettera di  $x$  è l'iniziale di  $y$



- i due giocatori si alternano formando, finché possono, un cammino semplice sul grafo

40

40

## Gioco GG della geografia generalizzato

- due giocatori **A** e **B** possono giocare il gioco seguente (inizia **A**)
  - un'istanza di gioco è una coppia  $G, s$  nota ad entrambi, dove  $G$  è un grafo orientato ed  $s$  è il vertice di partenza
  - **A** e **B** si alternano nell'estendere un cammino semplice  $p$  su  $G$  a partire da  $s$
  - il primo giocatore che non riesce ad estendere  $p$  ha perso
- ci si può chiedere: per una certa coppia  $G, s$  esiste una strategia che fa vincere **A**? se no vince **B**

41

41

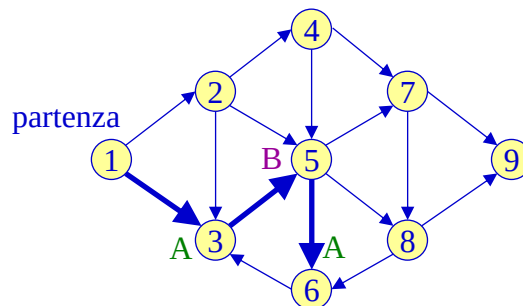
## Esiste una strategia che fa vincere il primo giocatore?

- a partire da  $s$
- esiste la possibilità per **A** di scegliere un vicino  $s_1$  di  $s$  tale che
- per ogni scelta da parte di **B** di un vicino  $s_2$  di  $s_1$
- esiste la possibilità di scegliere da parte di **A** un vicino  $s_3$  di  $s_2$
- ....

42

42

Esiste una strategia che fa vincere il primo giocatore?



43

43

Esiste una strategia che fa vincere il primo giocatore?

- data un'istanza di GG, il problema di stabilire se esista una strategia vincente per **A** su quell'istanza si può descrivere attraverso una formula nella quale le variabili di scelta sono tutte quantificate in modo esistenziale o universale
- l'appartenenza del problema a PSPACE si dimostra analogamente a quanto fatto per TQBF

44

44

## Algoritmo per decidere se A ha una strategia vincente

- input  $G, b$ :  $G$  grafo orientato e  $b$  vertice di partenza
  - se  $b$  non ha vicini allora rifiuta ( $A$  ha perso)
  - cancella  $b$  e tutti i suoi archi incidenti da  $G$
  - per tutti i vertici  $b_i$  originariamente vicini di  $b$  chiama ricorsivamente l'algoritmo su  $G, b_i$
  - se tutte le chiamate ricorsive accettano, allora rifiuta (vuol dire che  $B$  ha una strategia vincente)
  - altrimenti accetta ( $B$  non ha una strategia vincente quindi  $A$  ha una strategia vincente)

45

45

## Algoritmo per decidere se A ha una strategia vincente

- l'algoritmo usa spazio polinomiale
- infatti: l'unico spazio d'interesse è quello usato dallo stack per la ricorsione
  - ogni livello di ricorsione aggiunge allo stack un singolo vertice
  - il numero di livelli di ricorsione è limitato superiormente dal numero di vertici
- quindi lo spazio usato è lineare

46

46

## Esiste una strategia che fa vincere il primo giocatore?

- data un'istanza di GG, il problema di stabilire se esista una strategia vincente per A su quell'istanza è un problema PSPACE completo
- sono PSPACE completi analoghi problemi su altri giochi come go-moku (una variante del go), reversi, ....

47

47

## Differenza tra NP e PSPACE

- la classe NP è quella dei problemi le cui istanze positive hanno un certificato *corto*
- la classe PSPACE è quella delle strategie vincenti dei giochi a due giocatori *con conoscenza completa* (le mosse si fanno su una scacchiera completamente visibile a entrambi)
  - anche qui occorre esplorare uno spazio di ricerca di dimensioni esponenziali, ma qui un *certificato* di esistenza di una strategia vincente *sembra richiedere necessariamente uno spazio esponenziale*

48

48